PROJECT REQUIREMENTS

SupportDesk AI

ASP.NET Core Web API • AI Integration • Trainee Project

Purpose: assess whether developers with JavaScript experience can apply the .NET concepts covered in training to build a small, maintainable API and safely integrate an AI capability. The project is intentionally scoped for a short delivery window; a reliable, explainable MVP is more important than a large feature set.

1. Project brief

Build SupportDesk AI, a support-ticket management API. Users can create, view, update, filter, and close support tickets. An AI service can analyze a ticket and suggest its category, priority, sentiment, summary, and next action. It can also draft a response for an agent to review and edit. The system must never send AI-generated replies automatically. The application also exposes its ticket operations through a Model Context Protocol (MCP) server, so any MCP-compatible LLM client can use it.

2. Learning objectives

- Translate familiar JavaScript/API development concepts into C# and ASP.NET Core.
- Build REST endpoints with appropriate HTTP methods, status codes, validation, and error handling.
- Use dependency injection, interfaces, async/await, and a service layer.
- Persist data using Entity Framework Core and SQLite.
- Call an AI provider securely, validate structured output, and handle failures.
- Expose application capabilities to LLM clients through a Model Context Protocol (MCP) server.
- Demonstrate the application and explain the code and trade-offs without relying on memorized definitions.

3. Required technology

Backend	ASP.NET Core Web API using C# and controllers (or the approach agreed with the trainer)
Data access	Entity Framework Core
Database	SQLite for simple local setup
API exploration	Swagger / OpenAPI
AI	A hosted LLM API or Ollama local model, selected by the trainer/team
MCP server	MCP server using the official C# MCP SDK; stdio transport (streamable HTTP optional)
Tests	Unit tests for important business logic; API/manual test evidence
Frontend	Optional. Swagger is sufficient for the required MVP.

4. Functional requirements

FR-01 — Create ticket

POST /api/tickets. Accept a title and description. Validate required fields and reasonable length limits. Return the created ticket and an appropriate success status.

FR-02 — List tickets
GET /api/tickets. Return tickets. Support optional filters such as status, priority, and category if time allows.

FR-03 — Get ticket
GET /api/tickets/{id}. Return one ticket or 404 when it does not exist.

FR-04 — Update ticket
PUT /api/tickets/{id}. Allow permitted ticket fields to be updated. Validate input and return 404 for an unknown ID.

FR-05 — Update status
PATCH /api/tickets/{id}/status. Support Open, InProgress, Resolved, and Closed. Reject invalid status values.

FR-06 — AI analysis
POST /api/tickets/{id}/analyze. Return a suggested category, priority, sentiment, concise summary, and suggested action. Persist the result only after validating it.

FR-07 — Draft reply
POST /api/tickets/{id}/draft-reply. Generate a draft response based on the ticket. Return it for human review; do not send it.

FR-08 — Dashboard summary
GET /api/dashboard/summary. Return counts such as total, open, high-priority, and resolved tickets.

FR-09 — Failure handling
Return understandable error responses for validation errors, missing records, AI timeouts, provider errors, and malformed AI output. Do not expose secrets or stack traces.

FR-10 — Configuration
Read AI credentials and configuration from environment variables or user secrets, not committed source code.

FR-11 — MCP server
Expose SupportDesk AI as a Model Context Protocol (MCP) server so that any MCP-compatible LLM client can use the application. Build it with the official C# MCP SDK (the ModelContextProtocol NuGet package) as a separate project in the solution or as a startup mode of the API host. Support stdio transport; streamable HTTP transport is optional.

FR-12 — MCP tools
Expose these tools with clear names, descriptions, and typed input parameters: create_ticket, list_tickets, get_ticket, update_ticket, update_ticket_status, analyze_ticket, draft_reply, and get_dashboard_summary. Each tool calls the same service layer as the REST endpoints and must not duplicate business logic. Return structured results, and return controlled tool errors for validation failures, unknown IDs, AI timeouts, and provider errors.

FR-13 — MCP safety and client setup
Treat all tool arguments from the LLM client as untrusted input and validate them as strictly as API requests. The draft_reply tool returns a draft only and never sends anything. Do not expose delete operations, secrets, or stack traces through MCP. Document how to connect the server to an MCP client (for example Claude Desktop or MCP Inspector) with a sample client configuration, and show at least one end-to-end tool call in the demo.

5. Suggested data model

Ticket fields:

- Id (integer or GUID primary key)
- Title (required string), Description (required string)
- Status (Open, InProgress, Resolved, Closed)
- Priority (Low, Medium, High, Critical)
- Category and Sentiment (nullable until analyzed)
- AiSummary and SuggestedAction (nullable)
- CreatedAt and UpdatedAt (UTC timestamps)

Use enums or another explicit representation for fixed values. Do not let arbitrary AI text bypass validation for status or priority. Include an EF Core migration or clear database initialization instructions.

6. API contract examples

Create-ticket request (POST /api/tickets):

```
{
  "title": "Unable to access my account",
  "description": "I reset my password but still cannot sign in."
}
```

Example AI analysis response (POST /api/tickets/{id}/analyze):

```
{
  "category": "Account Access",
  "priority": "High",
  "sentiment": "Frustrated",
  "summary": "The customer cannot sign in after resetting their password.",
  "suggestedAction": "Check account status and the password reset flow."
}
```

The example is illustrative, not a requirement that the model always choose these values. Validate the AI response against a typed DTO/schema and allow values before returning or saving it.

7. Technical expectations

- Keep controllers thin. Put business logic in services.
- Register services through ASP.NET Core dependency injection. Use an IAiTicketService abstraction so the provider can be replaced or mocked.
- Use asynchronous I/O where appropriate and pass CancellationToken where practical.
- Use request/response DTOs rather than binding database entities directly to every endpoint.
- Use EF Core migrations or documented database creation steps.
- Keep secrets out of Git. Include an .env.example or configuration example with placeholder values only.
- Do not send real customer, confidential, or personal data to an external AI provider.
- Treat model output as untrusted input. Never execute generated code, SQL, commands, or actions.
- Treat arguments received from MCP clients as untrusted input and expose only the tools listed in FR-12. MCP tools call the existing services and contain no business logic of their own.
- If provider access is unavailable, use a clearly identified mock implementation for development, but demonstrate the real integration if required by the trainer.

8. Non-functional requirements

- A reviewer can clone the repository and run the project by following the README.
- Errors are consistent and useful; secrets and internal stack traces are not exposed.
- The code is organized, named consistently, and has no unnecessary architecture layers.
- Core ticket operations work without the AI provider being available.
- The MCP server starts from the documented steps and works with at least one MCP client.
- AI requests have a timeout and provider failures do not crash the API.
- At least the key business rules and AI-output validation have automated tests or repeatable test evidence.

9. Out of scope for this deadline

- Training or fine-tuning a model.
- RAG, vector databases, multi-agent orchestration, or complex workflows.
- Production-grade deployment, monitoring, billing, and scaling.
- Full authentication/authorization unless explicitly assigned.
- MCP authentication (OAuth), remote hosting of the MCP server, and multi-user MCP sessions.
- A polished frontend; a frontend is an optional extension after the API works.

10. Suggested work plan

Day	Target
Day 1	Create project, configure SQLite/EF Core, implement create/list/get ticket endpoints, verify persistence.
Day 2	Complete update/status operations, DTOs, validation, service layer, dependency injection, and consistent errors.
Day 3	Implement IAiTicketService, AI analysis, typed output validation, draft replies, secrets configuration, and timeout/error handling.
Day 4	Implement dashboard summary, the MCP server and tools (FR-11 to FR-13), tests, filtering if time permits, and fix integration bugs.
Day 5	Test from a clean clone, verify the MCP server from an MCP client, complete README (including MCP setup), prepare a short demo, and attend an individual code walkthrough.

11. Acceptance criteria / definition of done

The complete application is uploaded to GitHub, and the repository link is submitted to the trainer. The repository contains the source code and README, excludes secrets and generated build files, and is accessible to the reviewer.

- The project builds and starts using the documented steps.
- Ticket CRUD and status changes work and persist after restarting the application.
- Invalid requests are rejected with useful 4xx responses; unknown IDs return 404.
- AI analysis returns a valid, typed response or a controlled error; malformed output is not blindly saved.
- AI reply generation produces a draft only and requires a human to review it.
- The MCP server starts using the documented steps, lists the FR-12 tools, and an MCP client can call them successfully, including analyze and draft-reply.
- Provider credentials are not present in the repository.
- A README includes prerequisites, setup, configuration, migration/database steps, endpoint examples, MCP server setup with a sample client configuration, and known limitations.

- The trainee can explain the controller-to-service-to-database/AI flow and make a small change during review.

GitHub submission is mandatory: push the complete project to a GitHub repository and share the repository URL with the trainer by the deadline. If the repository is private, grant the trainer/reviewer access before submission. Never commit API keys, passwords, or other secrets.

12. Submission checklist

- Upload the complete project to GitHub and submit the repository URL. Include meaningful commits and ensure the reviewer has access.
- Source code and project/solution files
- README and setup instructions
- Database migration or initialization instructions
- Example requests/responses or exported API collection
- MCP server project, sample MCP client configuration, and evidence of tool calls from an MCP client (screenshot or log)
- Tests and evidence of key scenarios
- Short demo (live or recorded) and individual code walkthrough